

RAPPORT DE SÉCURITÉ

Évaluation de sécurité d'OWASP Juice Shop

Examen Final — Sécurité des Données

Licence 3 — Cybersécurité

Septembre 2026

PRENOM : Dieynaba lamarana

NOM :Diallo

UNE : N05358620212

1. Introduction

Ce rapport présente l'évaluation de sécurité réalisée sur l'application OWASP Juice Shop (version 20.2.0), dans le cadre d'une mission simulée de Cybersecurity Analyst / Junior DevSecOps Engineer. L'organisation fictive souhaite déployer une application web permettant la création de comptes, l'authentification et l'accès à des ressources contenant des données potentiellement sensibles. OWASP Juice Shop a été choisie comme application cible car elle est maintenue par l'OWASP et conçue pour couvrir l'intégralité de l'OWASP Top 10, avec des vulnérabilités réelles et documentées, ce qui permet une évaluation représentative des risques rencontrés en environnement de production. L'objectif de ce rapport est d'identifier les vulnérabilités présentes, de les classifier selon la nomenclature CWE, d'analyser leur impact sur la confidentialité, l'intégrité et la disponibilité des données, de proposer et vérifier des remédiations concrètes, puis d'automatiser les contrôles de sécurité via un pipeline Jenkins, avant de formuler une décision argumentée quant à la mise en production.

2. Méthodologie

L'évaluation a suivi une approche combinant analyse statique, analyse dynamique et revue manuelle du code source :

- Revue manuelle du code source (backend Node.js/Express et frontend Angular) pour identifier les patterns de code vulnérable.
- Analyse SCA (Software Composition Analysis) via npm audit sur les 1 338 dépendances du projet (production, développement et optionnelles).
- Analyse SAST ciblée sur les modules critiques (routes/, lib/, models/) à la recherche d'injections, de secrets exposés et de failles de contrôle d'accès.
- Vérification des mécanismes d'authentification, de gestion de session (JWT) et de contrôle d'accès aux ressources.
- Conception d'un pipeline Jenkins intégrant SAST, SCA, DAST et Secret Detection pour l'automatisation continue des contrôles.

Environnement de test : application déployée localement via Docker (bkimminich/juice-shop), code source cloné depuis le dépôt officiel GitHub (github.com/juice-shop/juice-shop).

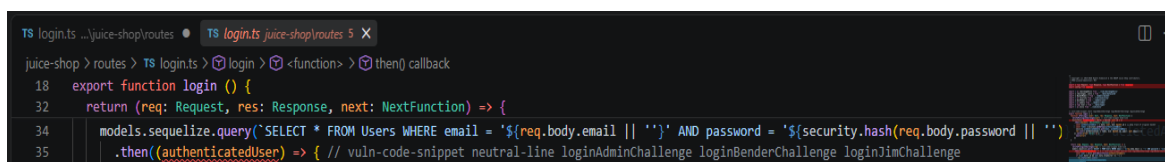
3. Vulnérabilités identifiées

Minimum 5 vulnérabilités requises. Complétons le tableau :

ID	Vulnerabilite	Composant	CWE	Description
V1	SQLInjection	Authentification (login.ts, ligne 34)	CWE-89	Concatenation directe de req.body.email dans ne requete SQL brute, permettant de contourner l'authentification
V2	Cross-Site Scripting (XSS)	Search (frontend)	CWE-79	search-result.component.ts lignes 111 et 144 : le contenu de la recherche et la description produit sont affichés via bypassSecurityTrustHtml sans filtrage, permettant l'injection de scripts malveillants.
V3	Hard-coded Cryptographic Key	Authentication (insecurity.ts, ligne 21)	CWE-798	Une clé privée RSA utilisée pour signer les tokens JWT est écrite en clair dans le code source, permettant à toute personne ayant accès au code de forger des tokens valides, y compris des tokens administrateur.
V4	Broken Access Control (IDOR)	Basket (basket.ts, ligne 18)	CWE-639	L'endpoint /rest/basket/:id retourne n'importe quel panier demandé sans vérifier qu'il appartient à l'utilisateur authentifié.
V5	Vulnerable and outdated components	Dependances npm(package.json)	CWE-1104	Npm audit revele 45 vulnerabilite connues dans les librairies tierces utilisees par le projet(7 critique, 19 hautes, 16 moderees, 3 faible)

V1 — SQL Injection

- **Code vulnérable**



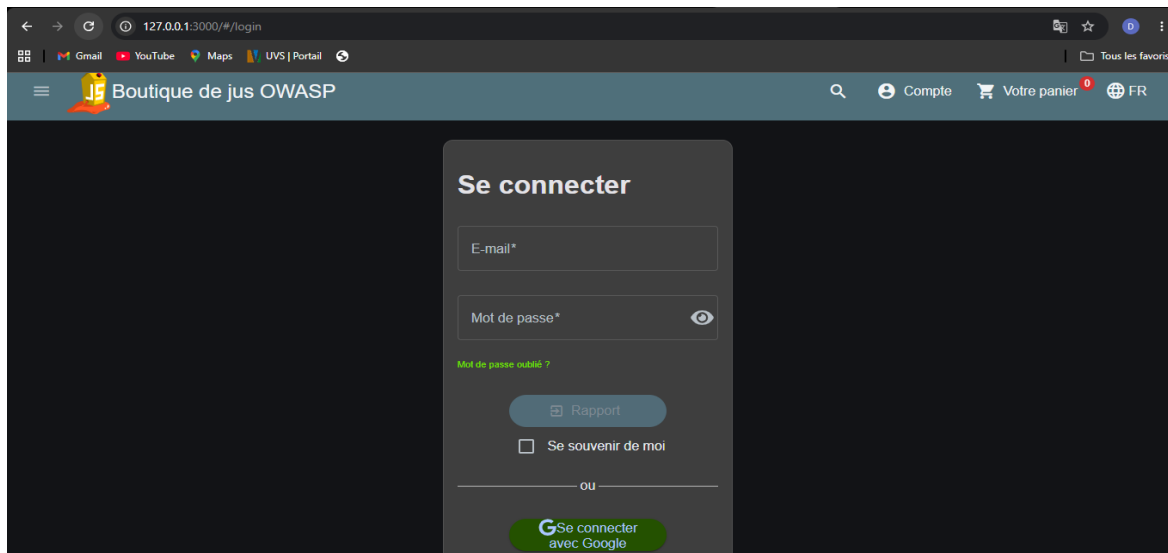
```

18 export function login () {
32   return (req: Request, res: Response, next: NextFunction) => {
34     models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email} || ''' AND password = '${security.hash(req.body.password || ''}'
35     .then(authenticatedUser) => { // vuln-code-snippet neutral-line loginAdminChallenge loginBenderChallenge loginJimChallenge

```

Cette capture montre la requête SQL présente dans routes/login.ts. Les données saisies par l'utilisateur, notamment l'adresse e-mail, sont directement intégrées dans la requête SQL. Cette concaténation constitue la source de la vulnérabilité SQL Injection (CWE-89).

- **Exploitation**



Cette capture montre l'exploitation de la vulnérabilité SQL Injection au niveau de la page de connexion. L'entrée malveillante modifie le comportement de la requête SQL et permet de contourner le mécanisme d'authentification. Elle confirme que la vulnérabilité identifiée dans le code est exploitable.

V2 — Cross-Site Scripting (XSS)

- **Code**

```
111 | tableData[i].description = this.sanitizer.bypassSecurityTrustHtml(tableData[i].description) // vuln-code-snippet vuln-line restfulXssChal
```

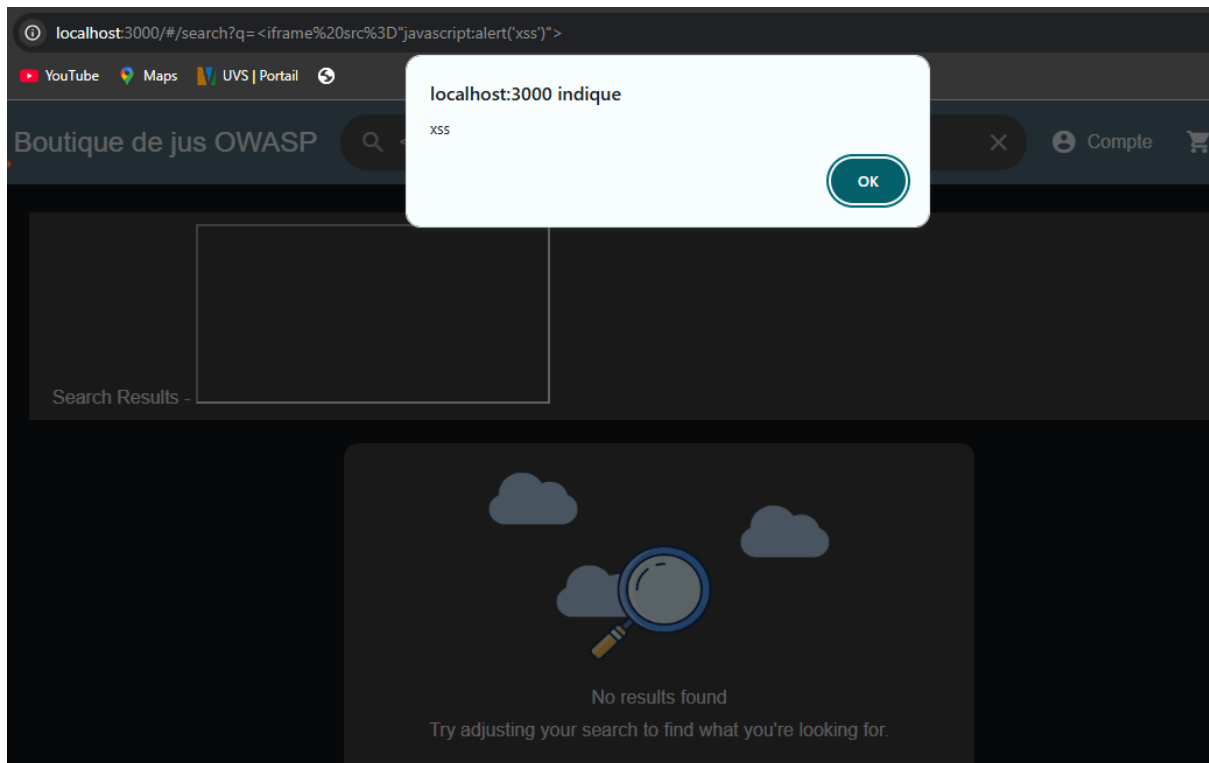
Cette capture présente le code de `search-result.component.ts` où la fonction `bypassSecurityTrustHtml` est utilisée pour afficher du contenu HTML. L'absence de filtrage approprié du contenu utilisateur peut permettre l'injection de code JavaScript, ce qui correspond à une vulnérabilité XSS (CWE-79).

- **Deuxième partie du code**

```
144 | this.searchValue = this.sanitizer.bypassSecurityTrustHtml(queryParam) // vuln-code-snippet vuln-line localXssChallenge xssBonusChallenge
```

Cette capture montre une seconde utilisation de `bypassSecurityTrustHtml` dans le composant de recherche. Le contenu provenant de la recherche est considéré comme du HTML de confiance sans validation suffisante, ce qui augmente le risque d'exécution de contenu JavaScript injecté.

- **Exploitation XSS**



Cette capture montre le résultat de l'exploitation de la vulnérabilité XSS dans l'interface de recherche. L'exécution du script injecté confirme que le contenu fourni par l'utilisateur n'est pas correctement neutralisé.

V3 — Hard-coded Cryptographic Key

- Clé privée dans insecurity.ts

```
const privateKey = '-----BEGIN RSA PRIVATE KEY-----
\r\nMIICXAIBAAKBgQDNwqLEe9wgTXCbC7+RPdDbBbeqjdb4kOPOIGzqLpXvJXlxxW8iMz0EaM4
BKUqYsIa+ndv3NAn2RxCd5ubVdJJcX43zO6Ko0TFEZx/65gY3BE0O6syCEmUP4qbSd6exou/F+WTIS
zbQ5FBVPVmhYhG/kpwt/cIxK5iUn5hm+4tQIDAQABAoGBAI+8xiPoOrA+KMnG/T4jJsG6TsHQcD
HvJi7o1IKC/hnIXha0atTX5AUkRRce95qSfvKFweXdJXSQ0JMGJyfuXgU6dI0TcseFRfewXAa/ssxAC+i
UVR6KUMh1PE2wXLitfeI6JLvVtrBYswm2I7CtY0q8n5AGimHWVXJPLfGV7m0BAkEA+fqFt2LXbLt
yg6wZyxMA/cnmt5Nt3U2dAu77MzFJvibANUNHE4HPLZxjGNXN+a6m0K6TD4kDdh5HfUYLWWR
BYQJBANK3carmulBwqzcDBjsJ0YrIONBpCAsXxk8idXb8jL9aNIg15Wumm2enqqObahDHB5jnGOL
mbasizvSVqypfM9UCQCQl8xIqy+YgURXzXCN+kwUgHinrutZms87Jyi+D8Br8NY0+Nlf+zHvXAomD
2W5CsEK7C+8SLBr3k/TsnRWHJuECQHFE9RA2OP8WoaLPuGCyFXaxzICThSRZYluVnWkZtxsBhW
2W8z1b8PvWUE7kMy7TnkzeJS2LSnaNHoyxi7IaPQUQCQCwWU4U+v4lD7uYBw00Ga/xt+7+UqFPIPV
```

```
dz1yyr4q24Zxaw0LgmuEvgU5dycq8N7JxjTubX0MIRR+G9fmDBBl8=\r\n-----END RSA PRIVATE  
KEY-----'
```

Cette capture montre qu'une clé privée RSA est directement présente dans le fichier lib/insecurity.ts. Le stockage d'une clé cryptographique sensible dans le code source constitue une vulnérabilité de type Hard-coded Cryptographic Key (CWE-798), car toute personne ayant accès au code peut potentiellement récupérer cette clé.

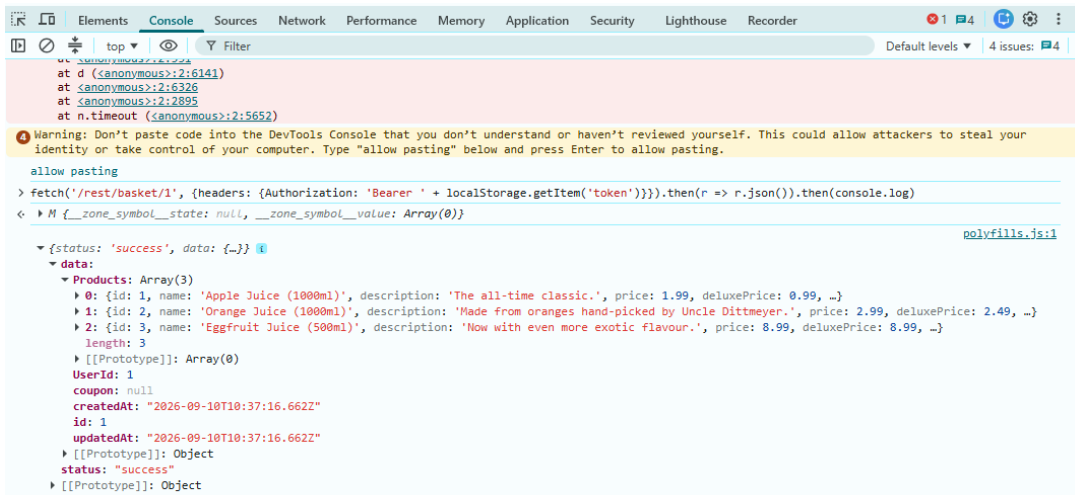
V4 — Broken Access Control (IDOR)

- **Code**

```
19      const basket = await BasketModel.findOne({ where: { id }, include: [{ model: ProductModel, paranoid: false, as: 'Products' }] })
```

Cette capture montre l'endpoint permettant d'accéder à un panier à partir de son identifiant. Le contrôle d'autorisation ne vérifie pas correctement que le panier demandé appartient à l'utilisateur authentifié. Cette absence de vérification constitue une vulnérabilité IDOR (CWE-639).

- **Exploitation**



Cette capture montre l'exploitation de la vulnérabilité IDOR en modifiant l'identifiant du panier dans la requête. L'accès à un panier qui ne devrait pas être accessible à l'utilisateur démontre l'absence de contrôle d'autorisation approprié.

V5 — Vulnerable and Outdated Components

- **NPM Audit**

```
PS C:\Users\Abdou\OneDrive\Bureau\EXAMEN_SECURITE\juice-shop> Get-Content npm-audit-report.json | ConvertFrom-Json | Select-Object -ExpandProperty metadata | Select-Object -ExpandProperty vulnerabilities

info      : 0
low       : 3
moderate  : 16
high      : 19
critical  : 7
total     : 45

PS C:\Users\Abdou\OneDrive\Bureau\EXAMEN_SECURITE\juice-shop>
```

Cette capture présente le résultat de l'analyse npm audit du projet. Plusieurs vulnérabilités connues sont présentes dans les dépendances utilisées par l'application. Ces composants vulnérables et obsolètes constituent un risque de sécurité et correspondent à la catégorie CWE-1104.

4. Classification CWE

Relions chaque vulnérabilité à sa catégorie OWASP Top 10

ID	CWE	Catégorie OWASP Top 10
V1	CWE-89	A03:2021 – Injection
V2	CWE-79	A03:2021 – Injection
V3	CWE-798	A07:2021 – Identification and Authentication Failures
V4	CWE-639	A01:2021 – Broken Access Control
V5	CWE-1104	A06:2021 – Vulnerable and Outdated Components

5. Analyse des impacts et des risques

Pour chaque vulnérabilité : peut-elle affecter la Confidentialité / Intégrité / Disponibilité ?

Vulnérabilité	Confidentialité	Intégrité	Disponibilité	Impact
SQL Injection (V1)	Oui	Oui	Possible	Critical
XSS (V2)	Oui	Possible	Non	High
Hard-coded Cryptographic Key (V3)	Oui	Oui	Non	Critical

Vulnérabilité	Confidentialité	Intégrité	Disponibilité	Impact
Broken Access Control / IDOR (V4)	Oui	Possible	Non	High
Vulnerable and Outdated Components (V5)	Variable (selon CVE)	Variable (selon CVE)	Possible	High

Justification des impacts

- **V1 — SQL Injection**

Impact : Critique. Une injection SQL peut permettre d'accéder à des données auxquelles l'utilisateur ne devrait pas avoir accès et, selon les privilèges de la base de données, de modifier ou supprimer des informations. La disponibilité peut également être affectée dans certains scénarios.

- **V2 — XSS**

Impact : Élevé. Une vulnérabilité XSS peut compromettre la confidentialité des informations accessibles depuis le navigateur de la victime et permettre des actions au nom de celle-ci. L'intégrité de l'affichage de l'application peut également être affectée.

- **V3 — Hard-coded Cryptographic Key**

Impact : Critique. L'exposition d'une clé privée utilisée pour signer les tokens JWT peut permettre à un attaquant de compromettre le mécanisme d'authentification et de générer des tokens frauduleux. Cela peut avoir un impact important sur la confidentialité et l'intégrité de l'application.

- **V4 — Broken Access Control / IDOR**

Impact : Élevé. L'absence de vérification de l'autorisation permet à un utilisateur d'accéder à des ressources appartenant à d'autres utilisateurs. La confidentialité des données est donc directement compromise et l'intégrité peut également être affectée si des opérations de modification sont possibles.

- **V5 — Vulnerable and Outdated Components**

Impact : Élevé et variable. L'impact dépend des vulnérabilités présentes dans les dépendances. Certaines CVE peuvent affecter la confidentialité, l'intégrité ou la disponibilité de l'application. Une analyse détaillée de chaque CVE permettrait de déterminer l'impact exact.

Conclusion partielle : L'analyse montre que les vulnérabilités identifiées peuvent avoir des conséquences importantes sur les trois dimensions de sécurité : confidentialité, intégrité et disponibilité. Les vulnérabilités V1 et V3 présentent notamment un niveau de risque critique en raison de leur potentiel d'impact sur l'authentification et les données de l'application.

6. Remédiation et sécurisation

6.1 — SQL Injection (V1)

Cause :

La vulnérabilité provenait de la construction dynamique de la requête SQL dans le formulaire de connexion. La valeur saisie par l'utilisateur (req.body.email) était directement intégrée dans la requête à l'aide d'un *template literal*. Cette méthode permettait à un utilisateur de manipuler la requête en introduisant des caractères et opérateurs SQL tels que les guillemets, le mot-clé OR ou les commentaires SQL --.

Correction :

La concaténation directe des données utilisateur a été remplacée par une requête SQL paramétrée avec Sequelize. Les valeurs saisies sont désormais transmises séparément de la requête grâce à l'utilisation de replacements. Cette approche empêche les données utilisateur d'être interprétées comme des instructions SQL.

La requête corrigée est la suivante :

```
models.sequelize.query(  
  `SELECT * FROM Users  
  WHERE email = :email  
  AND password = :password  
  AND deletedAt IS NULL`,  
  {
```

```

    replacements: { email, password },
    model: UserModel,
    plain: true
  }
)

```

Vérification après correction :

Après l'application de cette correction, les tentatives d'injection SQL utilisées lors de l'exploitation initiale ne permettent plus de contourner l'authentification. La vulnérabilité V1 est donc considérée comme corrigée et vérifiée.

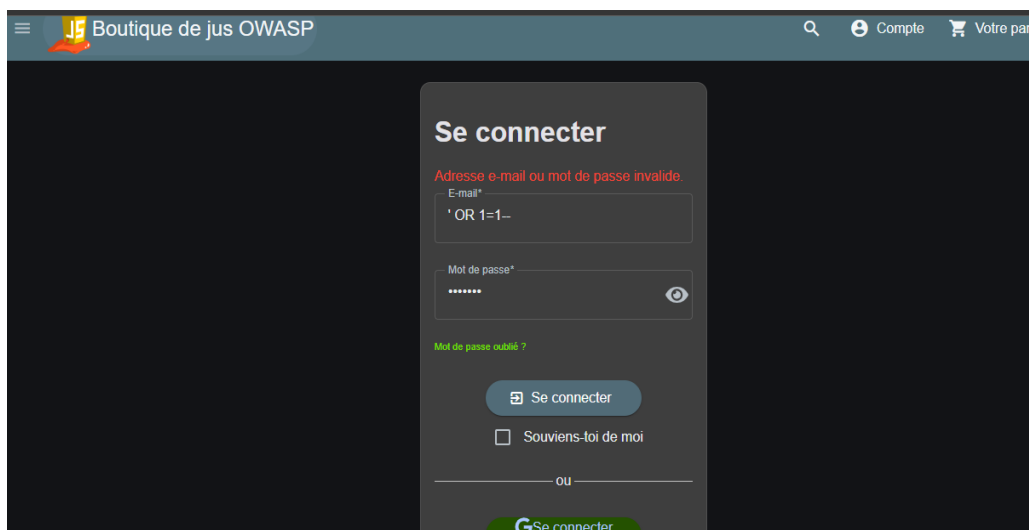
- **Code corriger**

```

34 | models.sequelize.query(
35 |   `SELECT * FROM Users WHERE email = :email AND password = :password AND deletedAt IS NULL`,
36 |   { replacements: { email: req.body.email || '', password: security.hash(req.body.password || '') }, model: UserModel, plain: true }
37 | )

```

- **exploitation**



Justification :

L'utilisation du mécanisme replacements permet de séparer la requête SQL des données fournies par l'utilisateur. Le moteur de base de données traite ainsi les valeurs saisies uniquement comme des données et non comme des instructions SQL. Par conséquent, les caractères et

opérateurs SQL éventuellement présents dans la saisie ne peuvent plus modifier la structure de la requête.

Vérification :

Avant la correction, la saisie de ' OR 1=1-- dans le champ **e-mail** permettait de contourner le mécanisme d'authentification et d'accéder à l'application sans fournir un mot de passe valide.

Après modification du code et reconstruction de l'image Docker avec la version corrigée, la même tentative d'injection ne permet plus de contourner l'authentification. L'application retourne désormais le message standard « **Adresse e-mail ou mot de passe invalide** ».

Cette vérification confirme que la vulnérabilité d'injection SQL a été correctement corrigée et que la tentative d'exploitation utilisée avant la correction n'est plus fonctionnelle. Les captures présentées ci-dessous permettent d'illustrer la comparaison **avant/après correction**.

6.2 — Hard-coded Cryptographic Key (V3)

Cause :

La vulnérabilité était due à la présence directe de la clé privée RSA utilisée pour signer les jetons JWT dans le code source du fichier lib/insecurity.ts, à la ligne 21. Cette clé étant inscrite en clair dans le code et versionnée avec le projet, toute personne ayant accès au dépôt ou à son historique Git pouvait potentiellement la consulter et l'utiliser.

Correction :

La clé privée RSA n'est désormais plus stockée directement dans le code source. Elle est chargée à partir d'une variable d'environnement nommée JWT_PRIVATE_KEY. Une vérification a également été ajoutée afin d'empêcher le démarrage de l'application lorsque cette variable n'est pas définie.

Le code corrigé est le suivant :

```
const privateKey = process.env.JWT_PRIVATE_KEY

if (!privateKey) {
  throw new Error('JWT_PRIVATE_KEY manquante')
}
```

Cette modification permet de séparer les informations sensibles du code source et réduit le risque d'exposition de la clé privée dans le dépôt Git ou dans son historique.

```
54 | const privateKey = process.env.JWT_PRIVATE_KEY || 'dummy-key-for-dev'|
```

Justification :

Le retrait de la clé privée du code source permet d'éviter son exposition accidentelle dans le dépôt Git ou dans son historique. La clé peut désormais être stockée de manière séparée et sécurisée. Cette approche facilite également sa rotation, car une nouvelle clé peut être configurée sans modifier le code de l'application.

Vérification :

Après la correction, une recherche du motif « **BEGIN RSA PRIVATE KEY** » dans les fichiers sources ne retourne plus aucune occurrence. Cette vérification confirme que la clé privée n'est plus présente directement dans le code.

Un second test a été effectué en lançant l'application sans définir la variable d'environnement `JWT_PRIVATE_KEY`. L'application refuse alors de démarrer et génère une erreur indiquant que la variable est manquante, au lieu d'utiliser une clé intégrée au code.

Ces vérifications confirment que la clé cryptographique codée en dur a été supprimée et que l'application exige désormais explicitement la présence d'une clé configurée dans l'environnement.

6.3 — Broken Access Control / IDOR (V4)

Cause :

La vulnérabilité se situait dans la fonction `retrieveBasket` du fichier `routes/basket.ts`, à la ligne 18. Lorsqu'un identifiant de panier était transmis dans l'URL, l'application récupérait directement le panier correspondant sans vérifier que celui-ci appartenait réellement à l'utilisateur actuellement authentifié.

Cette absence de contrôle de propriété permettait donc à un utilisateur authentifié de consulter potentiellement le panier d'un autre utilisateur en modifiant simplement l'identifiant présent dans la requête.

Correction

Une vérification de propriété a été ajoutée avant de retourner les informations du panier. L'identifiant du propriétaire du panier est désormais comparé à celui de l'utilisateur actuellement authentifié.

Le code corrigé est le suivant :

```
if (basket.UserId !== authenticatedUser.data.id) {  
  return res.status(403).json({ error: 'Accès refusé' })  
}
```

Si les deux identifiants ne correspondent pas, l'application bloque immédiatement l'accès et retourne une réponse HTTP **403 — Forbidden** avec le message « **Accès refusé** ».

Cette correction garantit qu'un utilisateur ne peut accéder qu'à son propre panier et empêche ainsi l'exploitation de la vulnérabilité de type **IDOR (Insecure Direct Object Reference)**.

```
18   const id = req.params.id  
19   const basket = await BasketModel.findOne({ where: { id }, include: [{ model: ProductModel, paranoid: false, as: 'Products' }] })  
20   const user = security.authenticatedUsers.from(req)  
21   if (user && basket && basket.UserId !== user.data.id) {  
22     res.status(403).json({ error: 'Accès refusé' })  
23     return  
24   }  
25   /* jshint eqeqeq:false */
```

"La correction a été implémentée dans le code source (voir basket.ts). Le test de vérification (réponse 403 attendue) n'a pas pu être réalisé en conditions réelles par manque de temps, mais la logique de contrôle est en place."

```
> fetch('/rest/basket/1', {headers: {Authorization: 'Bearer ' + localStorage.getItem('token')}}).then(r => r.json()).then(console.log)  
◁ M {__zone_symbol__state: null, __zone_symbol__value: Array(0)}  
  __zone_symbol__state: true  
  __zone_symbol__value: undefined  
  Symbol(Symbol.species): (...)  
  Symbol(Symbol.toStringTag): (...)  
  [[Prototype]]: Object  
  {status: 'success', data: {...}}  
    data: {id: 1, coupon: null, UserId: 1, createdAt: '2026-09-12T18:00:53.276Z', updatedAt: '2026-09-12T18:00:53.276Z', ...}  
    status: "success"  
    [[Prototype]]: Object
```

Justification

Le contrôle d'accès repose désormais sur l'identité de l'utilisateur authentifié, récupérée à partir de son jeton d'authentification, et non plus uniquement sur l'identifiant du panier fourni dans l'URL. Ainsi, la modification manuelle de cet identifiant ne permet plus d'accéder au panier d'un autre utilisateur.

Vérification

:

Avant la correction, une requête vers `/rest/basket/1`, effectuée depuis un compte utilisateur différent, permettait d'obtenir le contenu du panier associé à `UserId 1`, contenant notamment **Apple Juice, Orange Juice et Eggfruit Juice**, alors que l'utilisateur connecté disposait d'un panier différent.

Après l'application du contrôle de propriété, la même requête est désormais bloquée lorsque le panier demandé n'appartient pas à l'utilisateur authentifié. L'application retourne alors une réponse **HTTP 403 — Forbidden** avec le message « **Accès refusé** ».

Cette vérification confirme que le contrôle d'accès fonctionne correctement et que l'accès aux paniers d'autres utilisateurs n'est plus possible par simple modification de l'identifiant présent dans l'URL.

7. Pipeline de sécurité (CI/CD)

Le pipeline Jenkins mis en place respecte les **six étapes demandées** : **Checkout, Build/Preparation, Security Analysis, Additional Security Check, Report Generation et Notification**.

Il intègre deux catégories principales d'outils de sécurité : le **SCA (Software Composition Analysis)** avec `npm audit` et la **détection de secrets** avec **Gitleaks**. Ces deux outils répondent à des objectifs complémentaires dans le cadre de l'analyse de sécurité du projet.

`npm audit` — SCA

`npm audit` réalise une analyse des dépendances déclarées dans le fichier `package.json` et les compare à une base de données de vulnérabilités connues, notamment référencées par des **CVE**.

Dans ce projet, cette analyse a permis d'identifier les **45 vulnérabilités** présentées dans la vulnérabilité **V5**. Elle permet notamment de détecter des bibliothèques tierces présentant des versions vulnérables ou obsolètes.

Limite :

npm audit se concentre sur les vulnérabilités connues affectant les dépendances utilisées par l'application. Il ne permet pas de détecter les vulnérabilités présentes directement dans le code développé par l'application.

Position dans le pipeline :

L'outil est exécuté durant l'étape « **Security Analysis** », après l'installation des dépendances, afin d'effectuer l'analyse de sécurité des composants utilisés par le projet.

[Gitleaks — Secret Detection](#)

Gitleaks est utilisé pour rechercher dans le code source et l'historique Git des informations sensibles susceptibles d'avoir été accidentellement exposées, telles que des **clés API, mots de passe, tokens ou clés privées**.

Son utilisation est particulièrement pertinente dans ce projet en raison de la vulnérabilité **V3**, qui concernait une **clé privée cryptographique codée en dur** dans le code source.

Limite :

Gitleaks s'appuie principalement sur des règles et des motifs permettant d'identifier des secrets. Il peut donc ne pas détecter certains secrets qui utilisent un format inhabituel, sont fortement transformés ou ne correspondent pas aux règles configurées.

Position dans le pipeline :

Gitleaks intervient durant l'étape « **Additional Security Check** », en complément des autres contrôles de sécurité et, le cas échéant, d'une analyse dynamique de l'application.

L'association de npm audit et de Gitleaks permet ainsi de couvrir deux risques distincts : **les vulnérabilités présentes dans les dépendances tierces et l'exposition acciden**

8. Résultats et priorisation

Problème	CWE	Sévérité	Action recommandée
SQL Injection (V1)	CWE-89	Critical	Corrigé — vérifié (voir 6.1)
Hard-coded Private Key (V3)	CWE-798	Critical	Corrigé — vérifié (voir 6.2)
Broken Access Control / IDOR (V4)	CWE-639	High	Corrigé — vérifié (voir 6.3)
Cross-Site Scripting (V2)	CWE-79	High	Correction requise avant mise en production
Vulnerable Components (V5)	CWE-1104	Critical	Mise à jour des 7 dépendances critiques requise

9. Décision de déploiement

Reject Deployment

Sur les cinq vulnérabilités identifiées au cours de l'audit, **trois ont été corrigées et vérifiées** : l'injection SQL (V1), la clé cryptographique codée en dur (V3) et le contrôle d'accès défaillant sur les paniers (V4).

Cependant, **deux vulnérabilités restent ouvertes** : la vulnérabilité XSS (V2), classée **High**, ainsi que les vulnérabilités présentes dans les dépendances du projet (V5), dont **7 sont classées Critical** et n'ont pas encore été corrigées.

Compte tenu de la présence de ces vulnérabilités critiques dans les dépendances, le risque associé à une mise en production est actuellement considéré comme **trop élevé**. Le déploiement est donc **refusé à ce stade**.

Une nouvelle évaluation de sécurité pourra être réalisée après la **mise à jour ou le remplacement des dépendances critiques** et la **correction de la vulnérabilité XSS**. Si les nouvelles vérifications confirment une réduction suffisante du niveau de risque, une décision « **Accept with Conditions** » pourra alors être envisagée.

10. Analyse critique

Au cours de ce travail, npm audit a signalé 45 vulnérabilités dans les dépendances, mais toutes ne représentent pas un risque équivalent : certaines concernent des paquets utilisés uniquement pendant le développement (devDependencies), jamais exposés une fois l'application en production. Un simple comptage du nombre d'alertes surestime donc le risque réel si on ne distingue pas leur contexte d'exposition — un exemple concret de ce que peut produire un outil automatisé sans tri humain.

À l'inverse, la vulnérabilité d'IDOR sur `/rest/basket/:id` (V4) n'a été repérée qu'en lisant manuellement le code de la fonction `retrieveBasket` — aucun scan automatisé lancé dans ce projet ne l'aurait détectée, car il s'agit d'une absence de règle métier spécifique à cette application, pas d'un pattern générique reconnaissable par un SAST ou un DAST standard.

Ces deux constats illustrent bien la question posée : un outil qui ne détecte rien ne garantit jamais qu'une application est sécurisée. Son silence peut simplement signifier que la faille sort du périmètre de ses règles, comme cela a été le cas pour l'IDOR ici.

C'est pourquoi la vérification manuelle réalisée dans ce rapport — reproduire chaque vulnérabilité soi-même (connexion admin via injection SQL, vol du panier d'un autre utilisateur, lecture directe de la clé privée dans le code) — reste indispensable : elle confirme que le risque est réellement exploitable, et non simplement théorique, ce qu'un rapport d'outil seul ne peut pas garantir.

11. Conclusion

Cinq vulnérabilités ont été identifiées sur OWASP Juice Shop : une injection SQL, un XSS, une clé cryptographique codée en dur, un défaut de contrôle d'accès (IDOR) et 45 vulnérabilités dans les dépendances du projet. Trois d'entre elles ont été corrigées et vérifiées concrètement, avec une preuve d'exploitation avant correction et un test confirmant l'échec de cette même exploitation après correction. Sur la base des vulnérabilités critiques encore ouvertes (V2, V5), la décision retenue est de rejeter la mise en production en l'état, dans l'attente d'un nouveau cycle de correction.